Paying Back Duplication Debt in the Linux Kernel: How Code Clones Are Refactored in the Trenches

Anonymous Author(s)
Anonymous Affiliation(s)

***Abstract*—Code duplication is a well-known source of technical debt, but little is known about how developers of large, long-lived systems actually pay it back. We investigate code deduplication in the Linux kernel, where changes are publicly reviewed on mailing lists before they reach the upstream repository (mainline). Our goal is to understand how clones are refactored in practice, which refactoring techniques developers apply, and how maintainers receive deduplication patches. We combine two sources of evidence. First, through an ethnographic study, we submitted deduplication patches to the AMDGPU and IIO subsystems and followed their review process. We qualitatively analyzed 30 contributions, classifying the refactoring techniques applied and discussed with maintainers according to Fowler's refactoring catalog. Among our findings, we highlight that maintainers tend to resist deduplications that introduce new abstractions more than the reuse of existing ones. Second, we mined patches from the mailing lists for clone refactorings arriving in a sample of 71 contributions; we quantitatively analyzed the techniques applied to validate and expand the findings from our ethnographic study. Extract Function was the most frequent technique in both datasets, suggesting that even simple refactoring techniques can yield valuable contributions to a system as complex as the Linux kernel. By analyzing 101 contributions, this study provides empirical evidence of the refactoring techniques used to remove code duplication in the Linux kernel, bringing insight on how these techniques relate to the acceptance of deduplication during its review process.**

***Index Terms*—Linux kernel, Deduplication, Technical Debt, Refactoring**

I. Introduction

Developers often reuse existing code by copying, pasting, and adapting it, a practice that accompanies the evolution of large, long-lived systems [1]. The resulting duplication is a well-known code smell [2] and, as such, a form of technical debt [3], whose cost is paid during maintenance. A change to one copy, such as a bug fix, must be propagated to all the others, and inconsistent changes to clones that should evolve together are a known source of faults [4].

Linus's Law states that "given enough eyeballs, all bugs are shallow" [5]. In the Linux kernel, part of this examination takes place in a well-established and rigorous review process, in which patches are publicly reviewed on mailing lists before integration and only about a third of them are eventually accepted [6]. Extending this reasoning to technical debt, such a process would also be expected to catch duplicated code before it is merged, leaving few opportunities for simple deduplication. This expectation, however, does not seem to hold, as duplicated code, from small fragments to entire files and directories, is still found in the kernel [1].

In this context, we investigate how duplication debt is repaid in practice in the Linux kernel, examining which refactoring techniques developers use to remove clones and how maintainers respond to deduplication patches during review. To this end, we draw on two sources of evidence. First, we conducted an ethnographic study based on participant observation in the AMDGPU and IIO subsystems, in which we submitted deduplication patches ourselves and also followed the review of patches submitted by students during a university course. We qualitatively analyzed these 30 contributions, classifying, according to Fowler's refactoring catalog [2], the techniques they applied. Second, we mined the kernel mailing lists for clone refactorings, using a broad regular expression over duplication-related keywords that retrieved 207 threads, which we manually analyzed to retain only those actually concerned with code duplication, resulting in 71 contributions. We then quantitatively analyzed the refactoring techniques applied in these contributions, using the same catalog, to validate and expand the findings of the ethnographic study.

In the ethnographic study, maintainers were more reluctant to accept deduplications that introduced new abstractions than those that reused abstractions already available in the kernel. In both sources, Extract Function was the most frequently applied technique. These findings suggest that deduplication is a suitable entry point for contributors with less experience in the kernel, since, although it requires understanding the duplicated code and its context, it involves relatively low complexity and can often be addressed with simple refactoring techniques. They also indicate that, despite its rigorous review process, the Linux kernel still carries duplication debt that is cheap to repay. With this study, we aim to draw attention to the duplication debt that remains to be repaid in the Linux kernel and to offer insights into how it can be addressed.

II. Related work

Wang and Godfrey [7] searched Linux kernel commit messages for "duplicated code" and found that these intentional clone refactorings account for fewer than 1% of the clone changes detected in four subsystems. Choi et al. [8] examined how clones are refactored by comparing versions of three open-source Java projects, and found that refactored clone pairs were mostly handled with two patterns from Fowler's catalog, Replace Method with Method Object and Extract Method. Unlike these studies, which infer clone refactorings from commit messages or code versions, we examine deduplication patches together with their review, both from within

the community, by submitting and following patches, and on the kernel mailing lists, where the complete discussion of each contribution is available rather than only its commit message.

Other studies proposed ways of reducing duplication in the kernel. Kadav and Swift [9] found that 8% of driver code is very similar to other driver code and suggested procedural abstraction, better support for multiple chipsets, and table-driven programming to reduce it. Yu and Ramaswamy [10] classified 903 clone sets into three categories and suggested library functions, aspects, or no change, depending on the category. Kumar et al. [1] represented duplicated files and directories as templates in a layer above the code, reducing redundant code by 30–50% in the groups they studied. These techniques were proposed by researchers who analyzed the code from the outside, and none of these studies reports submitting the changes to kernel maintainers. In contrast, we study techniques applied in real patches submitted to kernel maintainers.

Arcanjo et al. [11] submitted deduplication patches to the AMDGPU and IIO subsystems and, from 13 patch series, identified four factors under which maintainers tolerate duplication, namely driver-forking, readability, integration overhead, and performance. In that study, the refactoring used in each contribution appears only as part of the account of each review, without being characterized or compared. We follow the same participant observation approach and expand the sample to 30 contributions, focusing on the techniques they apply and on how maintainers receive them. We also analyze contributions from the kernel mailing lists, to check whether the same techniques appear or whether new ones emerge.

III. Methodology

To understand how code duplication is removed in the Linux kernel, we combined two complementary sources of evidence. Through participant observation, we followed deduplication patches from their submission through their discussion with maintainers, which gave us an in-depth view of how these contributions are received. Through mining the kernel mailing lists, we collected deduplication contributions beyond those we followed, which allowed us to check whether the techniques observed in the first source also appear in a broader sample. We then classified the refactoring techniques in both samples according to Fowler's catalog [2], analyzing the first qualitatively and the second quantitatively. The following subsections describe each of these steps in detail.

A. Participant Observation

We adopted an ethnographic approach to observe how deduplication patches are received from within the kernel community, following each decision as it happens rather than reconstructing it afterwards from code artifacts. We conducted this participant observation in the IIO and AMDGPU subsystems of the Linux kernel, adopting two of the observer roles described by Gold [**?**], complete participant and participant as observer. In both roles, the duplications addressed by the patches were identified with ArKanjo [12], a tool for detecting function-level code duplication in the Linux kernel. We consider each patch series, including all its versions, as one contribution. In total, we qualitatively analyzed 30 contributions, 13 of which had been analyzed in a previous study [11] (C1 to C13) and 17 of which are new (C14 to C30), all listed in Table I.

In the complete participant role, we contributed to the kernel ourselves, selecting duplications reported by the tool, preparing deduplication patches, submitting them to the subsystem mailing lists, and interacting with maintainers during review, which resulted in 6 of the 17 new contributions.

In the participant as observer role, we followed contributions submitted by students during a university course in the first semester of 2026. We mentored them during the submission process and followed the reviews they received from maintainers. The students submitted the remaining 11 new contributions.

B. Mailing Lists Mining

To check whether the techniques observed in the participant observation also appear in contributions beyond those we followed, we mined e-mails from the kernel mailing lists available in the LKML5Ws dataset [13]. We used four lists, IIO, AMDGPU, IOMMU, and I2C.

Since the dataset stores individual e-mails, we first reconstructed the discussion threads, linking each e-mail to its parent through the `In-Reply-To` header, falling back to the last identifier in the `References` header when `In-Reply-To` was missing or pointed to a message absent from the dataset. We then grouped the linked e-mails into threads, and ordered the messages of each thread by date.

Based on the vocabulary that contributors and maintainers used to discuss duplication during the participant observation, we built a regular expression to filter threads. Among the threads that matched it, we randomly selected 220 threads. 13 threads were excluded, 11 that contained only pull requests, which summarize patches discussed in other threads and include neither diffs nor discussion, 1 that was already part of another thread in the sample, and 1 whose duplication terms appeared only in text quoted from other messages, leaving 207 threads.

Two authors independently labeled each of the 207 threads as related or not to code duplication, and resolved disagreements by consensus. This resulted in 71 threads related to code duplication, 25 from IOMMU, 18 from AMDGPU, 15 from IIO, and 13 from I2C, which form the sample we analyzed.

acrecentar referncia ao arquivo do sample e script

acho que falta a etapa que produz os resultados, como as series foram classificadas?

Talvez uma seção para discutir como interpretamos as técnicas do Fowler(voltadas para POO) para codigos em C (limitação) e porque usamos o catálogo do Fowler (eu mantive as classificações do david)
https://martinfowler.com/articles/refactoring-2nd-ed.html
"The reorientation towards a less class-centered view is a large part of this." (2nd edition)

IV. Results and Discussion

We classified the 30 patch series for code deduplication according to the refactoring techniques they apply, using Fowler's catalog [2]. Five techniques appear in our corpus (Table I): Extract Function is the most frequent, applied in 10 patch series, followed by Parameterize Function with 6, Substitute Algorithm with 5, Replace Inline Code with Function Call with 4, and Pull Up Method with 3. All five techniques are well-known entries of Fowler's catalog, which suggests that deduplication can be addressed without complex refactorings. These techniques differ in whether they reuse code that already exists in the kernel or introduce a new abstraction, and in where the shared code is placed. In the following subsections, we describe how each group of techniques is applied in the kernel and how code review shaped the final form of the submissions.

Most contributions were small. The median contribution removed 31 lines of code, and 27 of the 30 removed fewer than 100. Three contributions (C9, C11, and C29) even added between 1 and 4 lines. Size did not determine how a contribution was received: the three largest ones (C1, C2, and C17, each with more than 390 lines removed) were all accepted, and C17 was accepted in its first version despite changing 3,309 lines (+1,274 added, −2,035 deleted).

Objetivo da secao: as tecnicas devem ser usadas para mostrar que, de forma simples, e possivel resolver deduplicacoes (de forma nao complexa), para que outros contribuidores possam atuar.

* Simplificar a apresentacao das tecnicas: usa-las para caracterizar formas de resolver duplicacoes (extrair uma funcao, reutilizar uma implementacao existente, trocar codigo repetido por chamada de funcao, parametrizar), mostrando que sao mudancas localizadas, e evitar tratar cada tecnica como contribuicao em si ou gastar espaco com definicoes.
* Cruzar as tecnicas com os takeaways (e adicionar o takeaway 5), e comentar, como foi feito em "Introducing New Abstractions", que uma refatoracao e consequencia de um takeaway: "a consequence of the driver forking practice already observed in the subsystem".
* Alterar o termo takeaways. Sugestoes: Contextos ou Cenarios.

A. Reusing and Simplifying Existing Code

We classified techniques Substitute Algorithm and Replace Inline Code with Function Call as those that reuse or simplify existing code. Unlike the techniques in Section IV-B, they remove the duplication without adding a new layer to the code. Of the 30 patch series, only 9 fall into this group, that is, about 30%.

When the duplicated function already exists in another hardware version, the reuse happens through the function pointer tables that each version uses to register its callbacks. In contribution C20, the contributor found that `jpeg_v3_0` had an interrupt handler identical to the one in `jpeg_v2_0` and, following a maintainer's suggestion, simply pointed `jpeg_v3_0_irq_funcs` to the previous version's function, removing the duplicate. In this case, no new code is needed.

In the Substitute Algorithm patch series, the duplicated code was usually a sequence of `switch` cases that mapped values in a very similar way. Contributions C13, C17, and C23 replaced these `switch` cases with lookup tables, so the mapping is described as data instead of control flow. However, this change is not always seen as an obvious improvement. In contribution C23, a maintainer questioned whether the new table was really less prone to errors and concluded that the advantage was "rather the reduction in verbosity". Thus, a lookup table is accepted when it makes the mapping easier to read, as in C23, but C13 was rejected because the maintainer

TABLE I
Summary of Newcomer Deduplication Contributions[a].

ID	SS	SML	Refactoring Technique	Diff
C1	AMD	100%	Pull Up Method	+114 / −520
C2	AMD	90%	Pull Up Method	+90 / −489
C3	IIO	100%	–	+2 / −12
C4	IIO	100%	Extract Function	+16 / −20
C5	IIO	100%	Parameterize Function	+23 / −27
C6	IIO	100%	Parameterize Function	+12 / −30
C7	IIO	100%	Replace Inline Code with Function Call	+2 / −7
C8	IIO	90%	Parameterize Function	+17 / −70
C9	IIO	90%	Parameterize Function	+95 / −92
C10	IIO	90%	Extract Function	+20 / −36
C11	IIO	90%	Extract Function	+20 / −16
C12	IIO	90%	Replace Inline Code with Function Call	+149 / −243
C13	IIO	90%	Substitute Algorithm	+32 / −58
C14	AMD	100%	Extract Function	+86 / −114
C15	IIO	100%	Substitute Algorithm	+15 / −32
C16	IIO	100%	Substitute Algorithm	+71 / −119
C17	AMD	90%	Substitute Algorithm	+1,274 / −2,035
C18	AMD	100%	Extract Function	+41 / −124
C19	AMD	100%	Replace Inline Code with Function Call	+12 / −61
C20	AMD	100%	Replace Inline Code with Function Call	+13 / −42
C21	AMD	100%	Extract Function	+61 / −94
C22	AMD	100%	Extract Function	+56 / −102
C23	IIO	100%	Substitute Algorithm	+29 / −66
C24	IIO	100%	–	+42 / −88
C25	AMD	100%	Extract Function	+131 / −190
C26	AMD	100%	Extract Function	+42 / −75
C27	AMD	100%	Extract Function	+66 / −89
C28	AMD	90%	Pull Up Method	+47 / −124
C29	IIO	90%	Parameterize Function	+63 / −62
C30	IIO	90%	Parameterize Function	+42 / −60

[a] A table summarizing contributions, including subsystem (AMD or IIO), similarity threshold, refactoring techniques based on Fowler's refactoring catalog [2], and code differentials showing lines added and removed.

judged that the code reduction did not compensate for the loss of readability, which is directly related to scenario (T2) [11].

B. Introducing New Abstractions

The remaining techniques, Extract Function, Parameterize Function, and Pull Up Method, are those that somehow add a new layer to the code: a new function, a function moved to a shared file, or a more general function that combines the duplicated ones through parameters. These techniques were the most common in our corpus, with 19 of the 30 patch series, that is, about 63%.

For Extract Function, the main difference between the two subsystems is where the extracted function is placed. In the IIO contributions C4, C10, and C11, the duplicated functions belonged to the same driver, and the common function stayed in the same file. In AMDGPU, all Extract Function patch series dealt with functions duplicated across different hardware versions, a consequence of the driver forking (T1) practice already observed in the subsystem [11]. In these cases, the common function was moved to a generic layer shared by all hardware versions, such as `amdgpu_gfx.c` (contributions C21, C22, and C27) and `amdgpu_gmc.c` (contributions C25 and C26), or to a proposed new file (C18). This shows that, in AMDGPU, extracting a function also means choosing a place for it in the driver architecture, and, as discussed in Section IV-C, new files proposed by the patch series were removed during review.

Being identical, however, is not enough for a function to be moved to the generic layer. In contribution C22, the maintainers agreed that there was nothing specific to the hardware generation in the function and recognized that it was "probably just copy and pasted over when we brought up gfx12". In contribution C25, the duplicated functions were also identical, but the maintainer stated that they were intentionally kept separated for each generation. In practice, what decides if the code can be shared is whether its behavior belongs to a single hardware generation or to all of them, which the textual similarity alone does not show.

Parameterize Function appeared only in IIO, and how the parameter is added matters. In contribution C6, the two functions differed only in the type of `struct` they used, and the contributor passed a generic `struct` as a parameter, which was accepted. In C29, the common function received a boolean parameter (`is_accel`) and several levels of pointers, and a maintainer replied: "Here and below the function is completely uglified. Even kernel documentation suggests to avoid bool parameters like this." In C9, the parameter forced the reader to look both at the function and at where the value was set [11]. Both series were rejected for readability reasons (T2).

Pull Up Method differs from Extract Function in how much of the duplicated code becomes shared. In Extract Function, only part of a function becomes a new shared function, whereas in Pull Up Method the whole duplicated function is moved to a shared place. Contribution C2 illustrates this technique in the AMDGPU display code, where the `hpd_ack()` function was duplicated, with minimal changes, in the drivers of several GPU generations. Instead of extracting parts of these copies, the contributor replaced them with two library functions that covered all cases, so that each generation calls the shared implementation instead of keeping its own copy.

C. How Review Reshapes the Refactoring

A recurring pattern across some contributions is that maintainer review can redirect a patch, changing the abstraction scope, the refactoring technique, or both. Three contributions illustrate each of these outcomes.

In C18, the first version consolidated the duplicated functions into a new file, `amdgpu_common.c`, intended to group functions shared across the subsystem. A maintainer rejected this layout: "Please don't when they are functional identical then move them a layer up instead of messing up the backends." He then argued that each function would fit better in a different, already existing layer. Although the refactoring technique remained the same across all revisions, the abstraction scope shifted to follow the layering conventions already established in the subsystem.

In C27, the opposite happened. The scope was kept, but the technique changed. The contributor initially proposed a single generic function in `amdgpu_gfx.c` to replace `gfx_v11_0_handle_priv_fault` and `gfx_v12_0_handle_priv_fault`, arguing that the two were identical. In the first review, a maintainer pointed to a specific code fragment and noted "Even when they are identical on gfx11 and gfx12 this decoding here is HW specific and doesn't belong here". The target layer was not questioned, and `amdgpu_gfx.c` remained the destination across all revisions. As a result, the contributor switched from Pull Up Method to Extract Function. Therefore, only the generation-independent part was moved to `amdgpu_gfx.c`, while the hardware-specific decoding remained in each generation's own function.

In C19, review changed both the scope and the technique. The first version made `jpeg_v5_0_1.c` and `jpeg_v5_0_2.c` share a common interrupt routine placed in a new file, `jpeg_v5_0_interrupt.c`. This proposal was rejected in the first review because the routine was not common to the whole family and the maintainer pointed "This is not common to jpeg v5. jpeg_v5_0_0.c has a different interrupt handler". The maintainer then suggested a narrower alternative: "I would just make the implementation in jpeg_v5_0_1.c non-static and then just use it directly in jpeg_v5_0_2.c". The duplication was still removed, but the scope was changed from a family-wide file to the two drivers that actually share the behavior, and the technique changed accordingly, to Replace Inline Code with Function Call.

Although the outcomes are different, the main concern is the same in all three cases. The shared code either did not belong to the proposed layer or did not apply uniformly to every component that layer serves. These cases do not fit the four takeaways proposed in previous study [11]. However they relate to T1, since the objections derive from hardware-specific code, the maintainers did not defend the duplication itself, nor

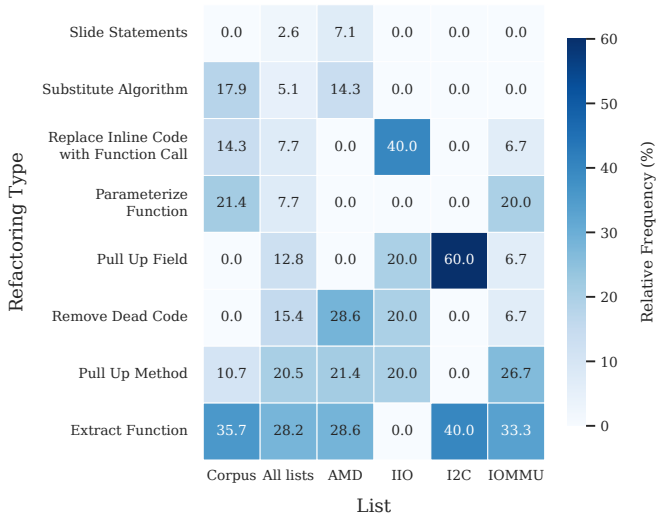


Fig. 1. Share of each refactoring technique among our submissions, all mailing lists, and each list. The value of n is the number of technique applications in each column.

did they invoke readability (T2), integration overhead (T3), or performance (T4). This leads to the fifth takeaway:

Takeaway T5 (Abstraction Scope): Maintainers may reject or redirect a deduplication when the duplicated code does not share an appropriate abstraction boundary. Consolidating such code into a shared layer can be considered inappropriate when the behavior does not apply uniformly to every component that the layer serves.

D. Deduplication in Other Kernel Contributions

To validate and expand the findings of our participant observation, we identified the refactoring techniques applied in the mailing-list series related to code duplication. Since a series may apply more than one technique, we count technique applications rather than series. Figure 1 compares the share of each technique among these applications with the share among our submissions, showing whether the techniques observed in our 30 contributions also appear beyond the ones we followed and which new ones emerge.

Extract Function was the most frequent technique in both sources. The techniques applied in our submissions also account for most of the applications in the mailing lists, including in IOMMU and I2C, where we did not submit patches. Thus, the techniques we applied are also used by contributors outside our group and outside the subsystems we observed, and they are basic entries of the catalog.

The mailing lists also revealed techniques that none of our submissions applied, namely Remove Dead Code, Pull Up Field, and Slide Statements. Pull Up Field moves fields repeated across structures to a shared one. Remove Dead Code was used both for unused code and for implementations that became redundant after equivalent functionality was made available elsewhere in the kernel, such as in a core layer.

Series without a catalog technique show that deciding whether to remove duplication is part of the task. Some introduced shared code to prevent future duplication rather than to remove an existing clone, and others tolerated duplication across distinct drivers, where abstracting would cost more than it would gain. In other cases, maintainers prevented duplication at its origin, asking the contributor to build on an existing driver instead of adding a new one. These cases are in line with the factors of the previous study [11].

Maintainers also took part in choosing the technique. In several series, a maintainer suggested a deduplication that the contributor had not proposed, such as parameterizing a function repeated several times or reusing an implementation already available in the kernel, as also happened in C19 and C20. Consequently, a contributor does not need to identify the right technique beforehand, since the review can point to it.

In summary, the mailing lists showed the same short list of basic techniques, led by Extract Function, in subsystems we did not observe, with maintainers helping to choose the technique. This supports the view that deduplication can be approached with simple, well-known refactorings.

E. Implications for New Contributors

The cases above can be read as a short guide for contributors with less experience in the kernel who want to remove duplication (Table II). Each row pairs a technique with a simple case we found and with what maintainers checked before accepting it. These contributions were made with a clone detection tool and, in part, with our mentoring, so they indicate an entry point into the kernel, and not a guarantee of acceptance.

References

[1] K. Kumar, S. Jarzabek, and D. Dan, "Managing big clones to ease evolution: Linux kernel example," in *Proceedings of the 2016 Federated*

TABLE II
How Refactoring Techniques Are Applied, and What to Consider[a].

Technique	Simple case	Check before sending
Repl. Inline Code	Equivalent function exists in another hardware version (C20).	Point the callback table to it, and confirm the behavior is the same.
Param. Func.	Functions differ only in a type (C6).	Avoid boolean parameters that send the reader back and forth (C29, C9).
Extract Func.	Fragment repeated across hardware versions (C27).	Keep hardware-specific code in each version (C27); use an existing layer, not a new file (C18); check that the duplication is not intentional (C25).
Subst. Algorithm	Repeated `switch` mapping values (C23).	Keep it only if readability holds (C13).
Pull Up Method	Whole identical functions across versions (C2).	Confirm the behavior applies to every version the layer serves (T5).

[a] Technique names are abbreviated.

Conference on Computer Science and Information Systems, ser. FedCSIS 2016, vol. 8. IEEE, Oct. 2016, pp. 1727–1736.

[2] M. Fowler, *Refactoring: Improving the Design of Existing Code*, 2nd ed. Boston: Addison-Wesley, 2018.

[3] P. Kruchten, R. L. Nord, and I. Ozkaya, "Technical debt: From metaphor to theory and practice," *IEEE Software*, vol. 29, no. 6, pp. 18–21, 2012.

[4] E. Juergens, F. Deissenboeck, B. Hummel, and S. Wagner, "Do code clones matter?" in *Proceedings of the 31st IEEE International Conference on Software Engineering (ICSE)*, 2009, pp. 485–495.

[5] E. S. Raymond, *The Cathedral and the Bazaar: Musings on Linux and Open Source by an Accidental Revolutionary*. O'Reilly Media, 1999.

[6] Y. Jiang, B. Adams, and D. M. German, "Will my patch make it? and how fast? case study on the Linux kernel," in *Proceedings of the 10th Working Conference on Mining Software Repositories (MSR)*, San Francisco, CA, USA, 2013, pp. 101–110.

[7] W. Wang and M. W. Godfrey, "Investigating intentional clone refactoring," in *Proceedings of the Eighth International Workshop on Software Clones*, ser. IWSC 2014, vol. 63. Electronic Communications of the EASST, 2014. [Online]. Available: https://eceasst.org/index.php/eceasst/article/view/2122

[8] E. Choi, N. Yoshida, and K. Inoue, "What kind of and how clones are refactored? a case study of three OSS projects," in *Proceedings of the Workshop on Refactoring Tools (WRT)*. Rapperswil, Switzerland: ACM, 2012.

[9] A. Kadav and M. M. Swift, "Understanding modern device drivers," in *Proceedings of the Seventeenth International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS '12. ACM, Mar. 2012, pp. 87–98.

[10] L. Yu and S. Ramaswamy, "Improving modularity by refactoring code clones: A feasibility study on Linux," *ACM SIGSOFT Software Engineering Notes*, vol. 33, no. 2, pp. 1–5, Mar. 2008.

[11] L. Arcanjo, D. Tadokoro, M. Spessoto, R. Passos, and P. Meirelles, "When do you repeat yourself? voices from the trenches of Linux kernel maintainers on code duplication," in *Proceedings of the 2026 ACM/IEEE International Conference on Technical Debt*, ser. TechDebt '26. ACM, Apr. 2026, pp. 47–52.

[12] L. Arcanjo, D. Tadokoro, and P. Meirelles, "ArKanjo: a tool for detecting function-level Code Duplication in the Linux Kernel," in *DebConf25*, O. Barais and B. Roucariès, Eds. Brest, France: IRISA, Jul. 2025, p. 3. [Online]. Available: https://hal.science/hal-05335545

[13] R. Passos, A. Pilone, D. Tadokoro, L. Arakaki, and P. Meirelles, "LKML5Ws: The what, when, who, where, and why in the Linux kernel mailing lists – a columnar dataset," Zenodo, 2026, dataset, version 1.0.0.